Failure (respecting a real rate limit) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The logic you're building is a small decision tree: given a response with `status_code == 429`, decide how long to wait before retrying.

- If `response.headers` has a `Retry-After` key, use that value (it's a string of seconds — convert it with `int(...)`).
- If it doesn't, fall back to your existing exponential backoff formula (`2 ** attempt`).

This is a real, common API convention — respecting `Retry-After` when a server provides it is considered good client behavior, not an optional nicety, because it's the server telling you exactly what it needs, instead of you guessing.

The exact pieces:

- **A minimal fake response** — you don't need `requests` at all for this exercise; a tiny class with just `status_code` and `headers` attributes is enough to test your logic in isolation.
- **`headers.get("Retry-After")`** — returns `None` cleanly if missing, which is exactly the signal you branch on.
- **Converting safely** — `Retry-After` is a string in the real header; wrap the `int(...)` conversion so a malformed value doesn't crash your retry logic.
- **Keeping this as its own function** — separate from your general backoff helper, so each piece of logic can be tested (and reasoned about) on its own.

Write the full function signature and body before checking Hint 2.

Two things a well-behaved client checks that "just read `Retry-After` and sleep" glosses over. First: the real HTTP spec allows `Retry-After` to be either a plain number of seconds (`"2"`) *or* an HTTP-date string (`"Wed, 21 Oct 2026 07:28:00 GMT"`) — a plain `int(retry_after)` crashes on the date form with a `ValueError` instead of falling back sensibly. A malformed value shouldn't

be able to crash your retry logic at all; it should just fall back to backoff, the same as if the header wasn't sent.

Second, and more important: a server is not necessarily trustworthy or even correct. A buggy or malicious server could send `Retry-After: 999999999` — should your client really sleep for over 31 years because a header told it to? A real client caps how long it's willing to wait on a server's say-so, the same way it caps how many times it's willing to retry at all. Blindly trusting an external number without a sanity check is the same class of mistake as blindly trusting a required config value without checking it — the fix is the same shape too: validate, then use, never just use.

The extra pieces:

- `email.utils.parsedate_to_datetime(value)` — parses the HTTP-date form of `Retry-After`; wrap it in `try/except` alongside the `int(...)` attempt, and fall back to backoff if neither parses.
- A hard cap, like `MAX_RETRY_AFTER_SECONDS = 60`, applied with `min(parsed_wait, MAX_RETRY_AFTER_SECONDS)` — never sleep longer than this, no matter what the header says.

Hint 2 — The plan, and almost the whole thing

```
class FakeResponse:
    status_code
    headers  # a dict

def decide_wait_seconds(response, attempt):
    retry_after = response.headers.get("Retry-After")
    if retry_after is not None:
        return int(retry_after)
    return 2 ** attempt

test:
    r1 = FakeResponse(429, {"Retry-After": "2"})
    decide_wait_seconds(r1, attempt=0)  # -> 2

    r2 = FakeResponse(429, {})
    decide_wait_seconds(r2, attempt=3)  # -> 8, from backoff
```

```
# retry_backoff_practice.py — Failure section
class FakeResponse:
    def __init__(self, status_code: int, headers: dict) -> None:
        self.status_code = status_code
        self.headers = headers

def decide_wait_seconds(response, attempt: int) -> int:
```

```
    retry_after = response.headers.get("Retry-After")
    if retry_after is not None:
        return int(retry_after)
    return 2 ** attempt
```

What's missing: the test calls for both branches, and type hints on `FakeResponse`'s use in the function signature. Write those yourself, then compare against the Solution's Approach 1.

Once that's working, go one step further — Approach 2, the shape the Build Task actually needs:

```
constant MAX_RETRY_AFTER_SECONDS = 60

function decide_wait_seconds(response, attempt):
    retry_after = response.headers.get("Retry-After")
    if retry_after is not None:
        try:
            wait = int(retry_after)
        except ValueError:
            try:
                parse retry_after as an HTTP-date, work out seconds from now
                wait = that many seconds
            except (bad date):
                wait = 2 ** attempt  # couldn't parse either form
        return min(wait, MAX_RETRY_AFTER_SECONDS)
    return 2 ** attempt
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# retry_backoff_practice.py — Failure section
from email.utils import parsedate_to_datetime
from datetime import datetime, timezone

MAX_RETRY_AFTER_SECONDS = 60

def decide_wait_seconds(response, attempt: int) -> float:
    retry_after = response.headers.get("Retry-After")
    if retry_after is None:
        return float(2 ** attempt)

    try:
        wait = float(retry_after)
    except ValueError:
        try:
            target_time = parsedate_to_datetime(retry_after)
            wait = (target_time - datetime.now(timezone.utc)).total_seconds()
            wait = max(wait, 0.0)
```

```
        except (TypeError, ValueError):
            wait = float(2 ** attempt)

    # your turn: don't return `wait` directly — cap it at
    # MAX_RETRY_AFTER_SECONDS using min(), so a server can't make you
    # sleep an absurd amount of time
    ...
```

Fill in the final `return` yourself, then compare all of your finished versions against the Solution.

Full solution: Show me the solution